

```

# =====
# TASK 1 — SQLAlchemy: Define Models and Connect
# =====

from sqlalchemy import (
    create_engine, Column, Integer, String, Date, Numeric,
    ForeignKey, UniqueConstraint
)
from sqlalchemy.orm import declarative_base, relationship

engine = create_engine(

    "postgresql+psycopg2://postgres:your_password@localhost:5432/college_db_orm",
    echo=True
)

Base = declarative_base()

class Department(Base):
    __tablename__ = "departments"

    dept_id = Column(Integer, primary_key=True, autoincrement=True)
    dept_name = Column(String(100), nullable=False, unique=True)
    head_of_dept = Column(String(100), nullable=False)
    location = Column(String(100))
    budget = Column(Numeric(12, 2))

    students = relationship("Student", back_populates="department")
    courses = relationship("Course", back_populates="department")
    professors = relationship("Professor", back_populates="department")

class Student(Base):
    __tablename__ = "students"

    student_id = Column(Integer, primary_key=True, autoincrement=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)

```

```
email = Column(String(100), nullable=False, unique=True)
dob = Column(Date, nullable=False)
dept_id = Column(Integer, ForeignKey("departments.dept_id"), nullable=False)
enrollment_year = Column(Integer)
```

```
department = relationship("Department", back_populates="students")
enrollments = relationship("Enrollment", back_populates="student")
```

```
class Course(Base):
    __tablename__ = "courses"

    course_id = Column(Integer, primary_key=True, autoincrement=True)
    course_name = Column(String(150), nullable=False)
    credits = Column(Integer, nullable=False)
    dept_id = Column(Integer, ForeignKey("departments.dept_id"), nullable=False)
    max_seats = Column(Integer, default=60)

    department = relationship("Department", back_populates="courses")
    enrollments = relationship("Enrollment", back_populates="course")
```

```
class Professor(Base):
    __tablename__ = "professors"

    professor_id = Column(Integer, primary_key=True, autoincrement=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    email = Column(String(100), nullable=False, unique=True)
    hire_date = Column(Date)
    dept_id = Column(Integer, ForeignKey("departments.dept_id"), nullable=False)
    salary = Column(Numeric(10, 2))

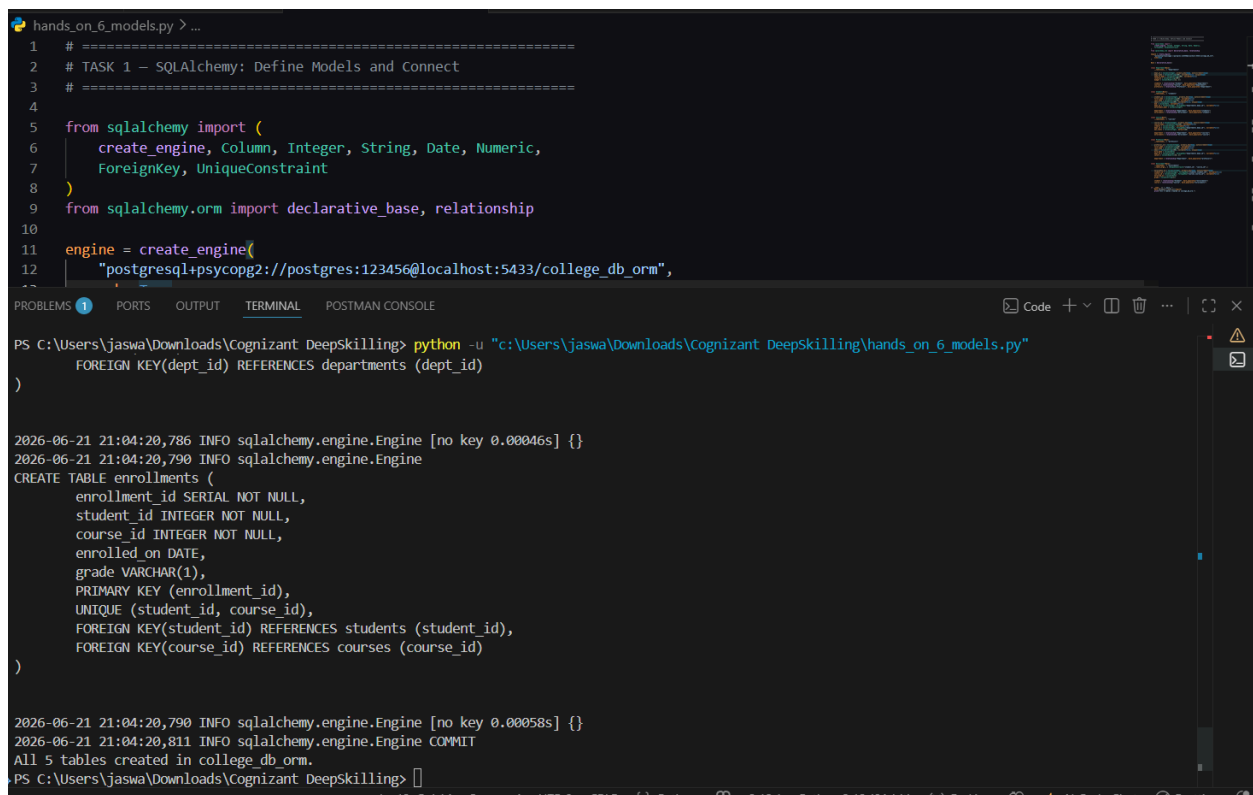
    department = relationship("Department", back_populates="professors")
```

```
class Enrollment(Base):
    __tablename__ = "enrollments"
    __table_args__ = (UniqueConstraint("student_id", "course_id"),)
```

```
enrollment_id = Column(Integer, primary_key=True, autoincrement=True)
student_id = Column(Integer, ForeignKey("students.student_id"), nullable=False)
course_id = Column(Integer, ForeignKey("courses.course_id"), nullable=False)
enrolled_on = Column(Date)
grade = Column(String(1))
```

```
student = relationship("Student", back_populates="enrollments")
course = relationship("Course", back_populates="enrollments")
```

```
if __name__ == "__main__":
    Base.metadata.create_all(engine)
    print("All 5 tables created in college_db_orm.")
```



The screenshot shows a code editor with a Python script named `hands_on_6_models.py`. The script defines a SQLAlchemy engine and creates a database with five tables. The terminal output shows the execution of the script, including the creation of the `enrollments` table and the message "All 5 tables created in college_db_orm."

```
hands_on_6_models.py > ...
1 # =====
2 # TASK 1 - SQLAlchemy: Define Models and Connect
3 # =====
4
5 from sqlalchemy import (
6     create_engine, Column, Integer, String, Date, Numeric,
7     ForeignKey, UniqueConstraint
8 )
9 from sqlalchemy.orm import declarative_base, relationship
10
11 engine = create_engine(
12     "postgresql+psycopg2://postgres:123456@localhost:5433/college_db_orm",
13 )
14
15 # =====
16 # TASK 2 - SQLAlchemy: Define Models and Connect
17 # =====
18
19 from sqlalchemy import Column, Integer, String, Date, Numeric,
20     ForeignKey, UniqueConstraint
21 from sqlalchemy.orm import declarative_base, relationship
22
23 Base = declarative_base()
24
25 class Student(Base):
26     __tablename__ = 'students'
27     student_id = Column(Integer, primary_key=True, autoincrement=True)
28     first_name = Column(String(50), nullable=False)
29     last_name = Column(String(50), nullable=False)
30     email = Column(String(100), nullable=False)
31     phone = Column(String(20), nullable=False)
32     address = Column(String(200), nullable=False)
33
34 class Course(Base):
35     __tablename__ = 'courses'
36     course_id = Column(Integer, primary_key=True, autoincrement=True)
37     course_name = Column(String(100), nullable=False)
38     credits = Column(Integer, nullable=False)
39
40 class Department(Base):
41     __tablename__ = 'departments'
42     dept_id = Column(Integer, primary_key=True, autoincrement=True)
43     dept_name = Column(String(50), nullable=False)
44
45 class Enrollment(Base):
46     __tablename__ = 'enrollments'
47     enrollment_id = Column(Integer, primary_key=True, autoincrement=True)
48     student_id = Column(Integer, ForeignKey('students.student_id'), nullable=False)
49     course_id = Column(Integer, ForeignKey('courses.course_id'), nullable=False)
50     enrolled_on = Column(Date, nullable=False)
51     grade = Column(String(1), nullable=False)
52
53 Base.metadata.create_all(engine)
54
55 if __name__ == "__main__":
56     Base.metadata.create_all(engine)
57     print("All 5 tables created in college_db_orm.")
```

```
PS C:\Users\jaswa\Downloads\Cognizant DeepSkilling> python -u "c:\Users\jaswa\Downloads\Cognizant DeepSkilling\hands_on_6_models.py"
2026-06-21 21:04:20,786 INFO sqlalchemy.engine.Engine [no key 0.00046s] {}
2026-06-21 21:04:20,790 INFO sqlalchemy.engine.Engine
CREATE TABLE enrollments (
  enrollment_id SERIAL NOT NULL,
  student_id INTEGER NOT NULL,
  course_id INTEGER NOT NULL,
  enrolled_on DATE,
  grade VARCHAR(1),
  PRIMARY KEY (enrollment_id),
  UNIQUE (student_id, course_id),
  FOREIGN KEY(student_id) REFERENCES students (student_id),
  FOREIGN KEY(course_id) REFERENCES courses (course_id)
)
2026-06-21 21:04:20,790 INFO sqlalchemy.engine.Engine [no key 0.00058s] {}
2026-06-21 21:04:20,811 INFO sqlalchemy.engine.Engine COMMIT
All 5 tables created in college_db_orm.
PS C:\Users\jaswa\Downloads\Cognizant DeepSkilling>
```

```
# =====
# Query count comparison (documented as required by Step 90):
#
# Task 2 Step 84 (lazy loading, default):
#   Querying all enrollments and accessing enrollment.student.first_name
#   and enrollment.course.course_name for each row triggers N+1 queries:
#   1 query to fetch all enrollments, then 1 additional query per
#   enrollment to lazy-load its related student, and 1 more per
#   enrollment to lazy-load its related course.
#   With 4 enrollments inserted in Step 82: 1 + 4 + 4 = 9 queries.
#   At larger scale (e.g. 13 enrollments from earlier hands-ons):
#   1 + 13 + 13 = 27 queries.
#
# Task 3 Step 88 (joinedload, eager loading):
#   session.query(Enrollment).options(
#       joinedload(Enrollment.student),
#       joinedload(Enrollment.course)
#   ).all()
#   issues exactly 1 SQL query with JOINS, regardless of row count.
#
# Result: query count drops from 9 (or 27) down to 1 — eliminating N+1.
# =====
```

```
from datetime import date
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, joinedload
from models import Base, Department, Student, Course, Enrollment, Professor

engine = create_engine(

    "postgresql+psycopg2://postgres:your_password@localhost:5432/college_db_orm",
    echo=True
)

Session = sessionmaker(bind=engine)
session = Session()

# =====
```

TASK 2 — CRUD Operations via ORM

=====

```
departments = [
    Department(dept_name="Computer Science", head_of_dept="Dr. Alan Turing",
        location="Block A", budget=750000.00),
    Department(dept_name="Mathematics", head_of_dept="Dr. Emmy Noether",
        location="Block B", budget=620000.00),
    Department(dept_name="Electronics", head_of_dept="Dr. Nikola Tesla",
        location="Block C", budget=580000.00),
]
session.add_all(departments)
session.commit()

cs_dept = session.query(Department).filter_by(dept_name="Computer
Science").first()
math_dept = session.query(Department).filter_by(dept_name="Mathematics").first()
ec_dept = session.query(Department).filter_by(dept_name="Electronics").first()

students = [
    Student(first_name="Alice", last_name="Johnson",
        email="alice.johnson@college.edu",
        dob=date(2001, 3, 15), dept_id=cs_dept.dept_id, enrollment_year=2021),
    Student(first_name="Bob", last_name="Smith", email="bob.smith@college.edu",
        dob=date(2000, 7, 22), dept_id=cs_dept.dept_id, enrollment_year=2021),
    Student(first_name="Carol", last_name="Williams",
        email="carol.williams@college.edu",
        dob=date(2002, 1, 10), dept_id=math_dept.dept_id, enrollment_year=2022),
    Student(first_name="David", last_name="Brown",
        email="david.brown@college.edu",
        dob=date(2001, 11, 5), dept_id=math_dept.dept_id, enrollment_year=2022),
    Student(first_name="Eva", last_name="Davis", email="eva.davis@college.edu",
        dob=date(2002, 6, 30), dept_id=ec_dept.dept_id, enrollment_year=2022),
]
session.add_all(students)
session.commit()

courses = [
```

```

    Course(course_name="CS101 - Intro to Programming", credits=4,
dept_id=cs_dept.dept_id),
    Course(course_name="CS201 - Data Structures", credits=4,
dept_id=cs_dept.dept_id),
    Course(course_name="MA101 - Calculus I", credits=3,
dept_id=math_dept.dept_id),
]
session.add_all(courses)
session.commit()

```

```

alice = session.query(Student).filter_by(email="alice.johnson@college.edu").first()
bob = session.query(Student).filter_by(email="bob.smith@college.edu").first()
carol = session.query(Student).filter_by(email="carol.williams@college.edu").first()

```

```

cs101 = session.query(Course).filter_by(course_name="CS101 - Intro to
Programming").first()
cs201 = session.query(Course).filter_by(course_name="CS201 - Data
Structures").first()
ma101 = session.query(Course).filter_by(course_name="MA101 - Calculus I").first()

```

```

enrollments = [
    Enrollment(student_id=alice.student_id, course_id=cs101.course_id,
        enrolled_on=date(2021, 8, 1), grade="A"),
    Enrollment(student_id=alice.student_id, course_id=cs201.course_id,
        enrolled_on=date(2021, 8, 1), grade="B"),
    Enrollment(student_id=bob.student_id, course_id=cs101.course_id,
        enrolled_on=date(2021, 8, 1), grade="B"),
    Enrollment(student_id=carol.student_id, course_id=ma101.course_id,
        enrolled_on=date(2022, 8, 1), grade="A"),
]
session.add_all(enrollments)
session.commit()

```

```

cs_students = (
    session.query(Student)
    .join(Department)
    .filter(Department.dept_name == "Computer Science")
    .all()
)

```

```

print("\n--- Students in Computer Science ---")
for s in cs_students:
    print(f"{s.first_name} {s.last_name}")

print("\n--- Enrollments (lazy loading — watch the SQL log for N+1) ---")
all_enrollments = session.query(Enrollment).all()
for e in all_enrollments:
    print(f"{e.student.first_name} {e.student.last_name} -> {e.course.course_name}")

student_to_update =
session.query(Student).filter_by(email="bob.smith@college.edu").first()
student_to_update.enrollment_year = 2023
session.commit()
print(f"\nUpdated {student_to_update.first_name}'s enrollment_year to
{student_to_update.enrollment_year}")

enrollment_to_delete = (
    session.query(Enrollment)
        .filter_by(student_id=carol.student_id, course_id=ma101.course_id)
        .first()
)
session.delete(enrollment_to_delete)
session.commit()
print("Deleted Carol's MA101 enrollment.")

remaining = session.query(Enrollment).count()
print(f"Remaining enrollments: {remaining}")

# =====
# TASK 3 — Eager Loading to Fix N+1
# =====

print("\n--- Enrollments (eager loading via joinedload — should be 1 query) ---")
eager_enrollments = (
    session.query(Enrollment)
        .options(
            joinedload(Enrollment.student),
            joinedload(Enrollment.course)
        )

```

```

    )
    .all()
)
for e in eager_enrollments:
    print(f'{e.student.first_name} {e.student.last_name} -> {e.course.course_name}')

session.close()

```

The screenshot shows a VS Code editor with a Python script named `hands_on_6_crud.py` and its terminal output. The script uses SQLAlchemy to connect to a database and query enrollment data. The terminal output shows the execution of the script, including the SQL query generated by SQLAlchemy and the resulting output of the program.

```

connect.py  hands_on_5.js 1  hands_on_6_models.py  hands_on_6_crud.py X
hands_on_6_crud.py > ...
26 import sys
27 from sqlalchemy import create_engine
28 from sqlalchemy.orm import sessionmaker, joinedload
29
30 CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
31 if CURRENT_DIR not in sys.path:
32     sys.path.insert(0, CURRENT_DIR)
33
34 from hands_on_6_models import Base, Department, Student, Course, Enrollment, Professor
35
36 engine = create_engine(
37     "nhostresal+nsvcone2://nhostres:123456@localhost:5433/college.db?orm="
)

```

```

PS C:\Users\jaswa\Downloads\Cognizant DeepSkilling> python -u "c:\Users\jaswa\Downloads\Cognizant DeepSkilling\hands_on_6_crud.py"
Remaining enrollments: 3

--- Enrollments (eager loading via joinedload - should be 1 query) ---
2026-06-21 21:13:55,608 INFO sqlalchemy.engine.Engine SELECT enrollments.enrollment_id AS enrollments_enrollment_id, enrollments.student_id AS enrollment
s_student_id, enrollments.course_id AS enrollments_course_id, enrollments.enrolled_on AS enrollments_enrolled_on, enrollments.grade AS enrollments_grade,
students_1.student_id AS students_1_student_id, students_1.first_name AS students_1_first_name, students_1.last_name AS students_1_last_name, students_1
.email AS students_1_email, students_1.dob AS students_1_dob, students_1.dept_id AS students_1_dept_id, students_1.enrollment_year AS students_1_enrollme
nt_year, courses_1.course_id AS courses_1_course_id, courses_1.course_name AS courses_1_course_name, courses_1.credits AS courses_1_credits, courses_1.de
pt_id AS courses_1_dept_id, courses_1.max_seats AS courses_1_max_seats
FROM enrollments LEFT OUTER JOIN students AS students_1 ON students_1.student_id = enrollments.student_id LEFT OUTER JOIN courses AS courses_1 ON courses
_1.course_id = enrollments.course_id
2026-06-21 21:13:55,608 INFO sqlalchemy.engine.Engine [generated in 0.00050s] {}
Alice Johnson -> CS101 - Intro to Programming
Alice Johnson -> CS201 - Data Structures
Bob Smith -> CS101 - Intro to Programming
2026-06-21 21:13:55,612 INFO sqlalchemy.engine.Engine ROLLBACK
PS C:\Users\jaswa\Downloads\Cognizant DeepSkilling>

```